

mcstatsim: Functional Monte Carlo simulation in R with process-based parallelism

Imad EL BADISY¹

¹ Independent researcher

DOI:

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Submitted:

Published:

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Summary

`mcstatsim` is an R package for Monte Carlo simulation studies that keeps the user-facing interface compact while supporting arbitrary user-supplied simulation functions. The core workflow takes a parameter grid, expands it across replications, and evaluates each task in parallel before binding the results into a tabular output that is straightforward to analyze downstream.

The package is designed for heavy simulation workloads where process-based parallelism has enough work to amortize cluster overhead. That design choice is important because many simulation studies need to call ordinary R code rather than compiled code, so the backend must remain compatible with arbitrary functions supplied by the user.

Statement of need

Simulation studies in statistics often involve repeated execution of an R function over a parameter grid, followed by computation of estimators, bias, coverage, or error metrics. In practice, the main engineering burden is not the statistical calculation itself, but the orchestration of many tasks, the collection of outputs, and the safe use of parallel workers.

`mcstatsim` addresses that problem by providing a functional interface centered on `runsim()` and `mcpmap()`. The package accepts arbitrary R simulation functions, preserves a visible progress bar during execution, and returns data-frame outputs that are easy to summarize with standard R tools.

Implementation

The current backend uses base `parallel` with a task-oriented execution model. Tasks are prepared once, dispatched in chunks, and combined with `data.table::rbindlist()` (Barrett et al. 2026) to reduce binding overhead. On Unix-like systems, the package uses a forked fast path when progress reporting is disabled; otherwise it falls back to a PSOCK cluster so the same code works across platforms.

This architecture keeps the implementation in R, which is the right trade-off for a package whose primary objective is to safely parallelize arbitrary user-supplied simulation functions. It also keeps the progress bar on the master process, so progress reporting remains usable while workers are busy.

Benchmark results

We compared `mcstatsim` against two commonly used alternatives for parallel apply-style workflows: `pbapply` (Solymos and Zawadzki 2025) and `future.apply` (Bengtsson

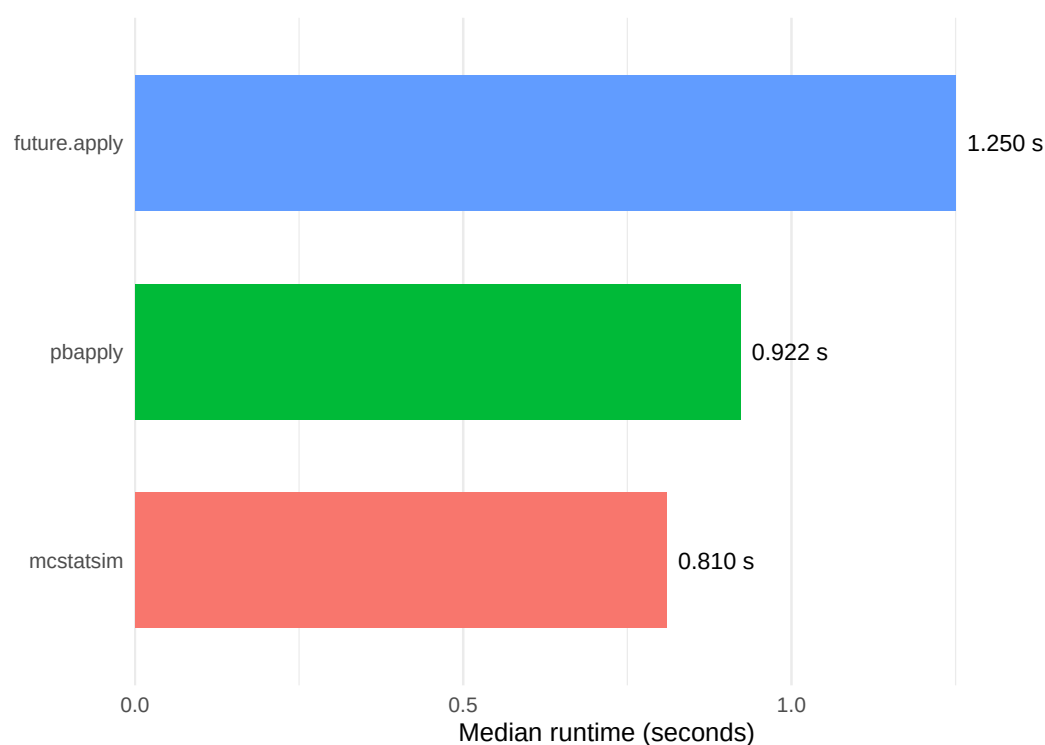


Figure 1: Median benchmark runtime for the compared backends on the synthetic Monte Carlo workload.

2021). The benchmark used a heavy synthetic Monte Carlo workload with 60 replications over a 16-cell parameter grid, evaluated on four workers. Timing was measured with `bench::mark()` (Hester and Vaughan 2025) using memory profiling disabled for the parallel cases.

The benchmark results are shown in Table 1 and Figure 1. On this workload, `mcstatsim` had the shortest median runtime.

Table 1: Benchmark comparison on the heavy synthetic workload used during development.

Backend	Median runtime (s)	Runtime vs. <code>mcstatsim</code>	Gain vs. <code>mcstatsim</code>
<code>mcstatsim</code>	0.810	1.00x	0%
<code>pbapply</code>	0.922	1.14x	12.1%
<code>future.apply</code>	1.250	1.54x	35.2%

On this benchmark, `mcstatsim` reduced median runtime by 12.1% relative to `pbapply` and by 35.2% relative to `future.apply`. These results are workload dependent, but they show that a task-oriented backend with chunked process-based parallelism can provide a measurable advantage for heavy Monte Carlo studies.

Limitations

`mcstatsim` is not intended to accelerate very small jobs. As with other process-based parallel approaches in R, cluster startup and serialization costs can dominate when each simulation task is too small. The package is therefore best suited to simulation studies with enough computation per task to justify parallel dispatch.

Acknowledgements

This work uses R (R Core Team 2025) and several community-developed packages. The benchmark comparison was informed by `pbapply` (Solymos and Zawadzki 2025), `future.apply` (Bengtsson 2021), `data.table` (Barrett et al. 2026), and `bench` (Hester and Vaughan 2025).

References

- Barrett, Tyson, Matt Dowle, Arun Srinivasan, Jan Gorecki, Michael Chirico, Toby Hocking, Benjamin Schwendinger, and Ivan Krylov. 2026. *Data.table: Extension of Data.frame*. <https://doi.org/10.32614/CRAN.package.data.table>.
- Bengtsson, Henrik. 2021. “A Unifying Framework for Parallel and Distributed Processing in `r` Using Futures.” *The R Journal* 13 (2): 208–27. <https://doi.org/10.32614/RJ-2021-048>.
- Hester, Jim, and Davis Vaughan. 2025. *Bench: High Precision Timing of `r` Expressions*. <https://doi.org/10.32614/CRAN.package.bench>.
- R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org/>.
- Solymos, Peter, and Zygmunt Zawadzki. 2025. *Pbapply: Adding Progress Bar to `'*Apply'` Functions*. <https://doi.org/10.32614/CRAN.package.pbapply>.